

Spring Boot Actuator



Spring Actuator

Spring Actuator is a part of the Spring Boot framework that helps you monitor and manage your application when it's pushed to production.

You can use it to get various production-ready features straight out of the box, such as health checks, metrics, info, and environment properties.



Actuator

Health Checks: It provides health status of the application which can be crucial for checking the status of back-end services like database, message broker, etc.

Metrics: It can gather and export metrics from your application so you can monitor things like memory usage, thread details, garbage collection, and HTTP requests.

Environment Properties: You can access and manage configuration and environment properties, which helps in understanding how applications are configured in different environment.

Log Configuration: Adjust the logging levels of your application dynamically at runtime without needing a restart.

Thread Dumps: It provides details of thread state in your application, which can be useful for debugging and monitoring purposes

HTTP Traces: It shows HTTP trace data for the requests made to the application, which is useful for tracing issues and performance.



Auditing: You can capture and expose audit events like login attempts or data access.

Add actuator dependency

```
<dependency>  
    <groupId>org.springframework.boot</  
groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
    </dependency>
```



Change application properties

All configurations under actuator are under management The following line can be added to the application.properties file:

```
management.endpoint.health.show-details=always
```



Manage its security:SecurityConfig

Since the actuator will expose sensitive information, manage its security appropriately:

```
.authorizeHttpRequests(auth ->  
auth.requestMatchers("/actuator","/actuator/**").permitAll())
```

For production this should be under strong security for authenticated

```
.authorizeHttpRequests(auth ->  
auth.requestMatchers("/actuator","/actuator/**").authenticated())
```



Preview

1

<http://localhost:8095/actuator>



```
{"_links":{"self":  
{"href":"http://localhost:8095/actuator","templated":false},"health":  
{"href":"http://localhost:8095/actuator/health","templated":false},"health-  
path":  
{"href":"http://localhost:8095/actuator/health/{*path}","templated":true}}}
```

2

[http://localhost:8095/actuator/
health](http://localhost:8095/actuator/health)



```
{"status":"UP","components":{"db":{"status":"UP","details":  
{"database":"PostgreSQL","validationQuery":"isValid()"}},"diskSpace":  
{"status":"UP","details":  
{"total":583124074496,"free":370616655872,"threshold":10485760,"path":"/home/  
damascene10/springboot/mysecurity/.","exists":true}},"ping":{"status":"UP"}}}
```



Monitor and expose everything or few components

When you want to monitor everything or some components you can add to one of the following line:

#Everything

```
management.endpoints.web.exposure.include=*
```

#Some components

```
management.endpoints.web.exposure.include=prometheus,health,env
```

localhost:8095/actuator/env

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8095/actuator",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8095/actuator/beans",
      "templated": false
    },
    "caches": {
      "href": "http://localhost:8095/actuator/caches",
      "templated": false
    },
    "caches-cache": {
      "href": "http://localhost:8095/actuator/caches/{cache}",
      "templated": true
    },
    "health": {
      "href": "http://localhost:8095/actuator/health",
      "templated": false
    },
    "health-path": {
      "href": "http://localhost:8095/actuator/health/{*path}",
      "templated": true
    },
    "info": {
      "href": "http://localhost:8095/actuator/info",
      "templated": false
    },
    "conditions": {
      "href": "http://localhost:8095/actuator/conditions",
      "templated": false
    },
    "configprops": {
      "href": "http://localhost:8095/actuator/configprops",
      "templated": false
    },
    "configprops-prefix": {
      "href": "http://localhost:8095/actuator/configprops/{prefix}",
      "templated": true
    },
    "env": {
      "href": "http://localhost:8095/actuator/env",
      "templated": false
    }
  }
}
```



You can include few features

```
management.endpoints.web.exposure.include=prometheus,health
```



Preview environment variables: <http://localhost:8095/actuator/env>

localhost:8095/actuator/env

```
{
  "activeProfiles": [],
  "defaultProfiles": ["default"],
  "propertySources": [
    {
      "name": "server.ports",
      "properties": {
        "local.server.port": {
          "value": "*****"
        }
      },
      "name": "commandLineArgs",
      "properties": {
        "spring.output.ansi.enabled": {
          "value": "*****"
        }
      },
      "name": "servletContextInitParams",
      "properties": {},
      "name": "systemProperties",
      "properties": {
        "java.specification.version": {
          "value": "*****"
        },
        "sun.jnu.encoding": {
          "value": "*****"
        },
        "java.class.path": {
          "value": "*****"
        },
        "com.sun.management.jmxremote.authenticate": {
          "value": "*****"
        },
        "java.vm.vendor": {
          "value": "*****"
        },
        "sun.arch.data.model": {
          "value": "*****"
        },
        "java.vendor.url": {
          "value": "*****"
        },
        "catalina.useNaming": {
          "value": "*****"
        },
        "user.timezone": {
          "value": "*****"
        },
        "org.jboss.logging.provider": {
          "value": "*****"
        },
        "java.vm.specification.version": {
          "value": "*****"
        },
        "os.name": {
          "value": "*****"
        },
        "sun.boot.library.path": {
          "value": "*****"
        }
      }
    }
  ]
}
```

Editor env setting
and java env
settings

Different
environment
variables from
application
properties

```
Environment Property \ name \ }}, { name : config resource class path
resource [application.properties] via location
'optional:classpath:/', "properties": {
  "spring.application.name": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 1:25"
  },
  "spring.main.allow-circular-references": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 2:39"
  },
  "spring.jpa.hibernate.ddl-auto": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 3:33"
  },
  "server.port": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 4:13"
  },
  "spring.datasource.username": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 5:28"
  },
  "spring.datasource.password": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 6:28"
  },
  "spring.jpa.hibernate.naming.physical-strategy": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 7:47"
  },
  "spring.datasource.url": {
    "value": "*****",
    "origin": "class path resource [application.properties] - 8:13"
  }
}
```

Metrics

Check metrics using

<http://localhost:8095/actuator/metrics>

Or individual metric

<http://localhost:8095/actuator/metrics/jdbc.connections.idle>

```
{"name":"jdbc.connections.idle","description":"Number of established but idle connections.", "measurements":[{"statistic":"VALUE","value":10.0}], "availableTags":[{"tag":"name", "values":["dataSource"]}]}
```



Micrometer dependency

Prometheus is a dimensional time series database with a built-in UI, a custom query language, and math operations.

Prometheus is designed to operate on a pull model, periodically scraping metrics from application instances, based on service discovery.

```
<!--a metrics instrumentation library designed for Java-based application-->
```

```
  <dependency>
```

```
    <groupId>io.micrometer</groupId>
```

```
  <artifactId>micrometer-registry-prometheus</artifactId>
```

```
  </dependency>
```



Check metrics

Check the url results: <http://localhost:8095/actuator/prometheus>

Example: Logs

TYPE logback_events_total counter

logback_events_total{level="debug"} 7765.0

logback_events_total{level="error"} 0.0

logback_events_total{level="info"} 5.0

logback_events_total{level="trace"} 627.0

logback_events_total{level="warn"} 0.0



Monitor Logs: Add the dependency

`<dependency>`

`<groupId>net.logstash.logback</groupId>`

`<artifactId>logstash-logback-encoder</`

`artifactId>`

`<version>7.4</version>`

`</dependency>`

Levels:

`["OFF","ERROR","WARN","INFO","DEBUG","TRACE"]`

Changing
level

`logging.level.root=warn`



Read more

<https://docs.spring.io/spring-boot/docs/2.1.7.RELEASE/reference/html/production-ready-metrics.html>

